Day 13 Evidence Workbooklet

Break, Continue, and Loop Debugging

Engineering practice to scan loop to skip or stop to expected-vs-actual evidence

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/50		Mastery check	secure / developing / needs support	

Concrete engineering situation

A checkout scan may skip incomplete records or stop when a severe safety issue appears. Day 13 introduces break and continue as loop-control moves that must be justified with trace evidence.

Engineering task	Trace a repeated checkout scan that either skips a record, stops early, or continues to the next record.
Computational move	Distinguish normal iteration from continue-skipped work and break-terminated loops.
Python focus	for loop scan, break, continue, skipped statements, early termination, debug trace, and protective tests.
Evidence to keep	current record, condition result, continue or break decision, skipped code, final count, and expected-vs-actual mismatch.

Day 13 working rules

1. continue skips the rest of the current pass and moves to the next pass.
2. break stops the whole loop immediately.
3. Statements after continue or break may not run.
4. Trace the current record before deciding skip or stop.
5. Debug loop-control bugs with expected and actual processed records.

Evidence map **Manage your evidence** blueprint

Part	Section	Evidence focus
1	Read continue as skip this pass	recognize, skip
2	Trace break as stop the whole scan	trace, stop
3	Distinguish skipped from stopped	compare, explain
4	Debug wrong loop-control placement	debug, test
5	Choose break, continue, or normal pass	decide, implement
6	Transfer and support route	transfer, reflect

1 Read continue as skip this pass

recognize skip

Incomplete records should not be counted as reviewed. The loop skips them.

```
reviewed = 0

for status in ["ready", "", "hold"]:
    if status == "":
        continue
    reviewed = reviewed + 1

print(reviewed)
```

Pts.	Prompt	My evidence / response
1	Which status value triggers continue?	
1	Which line is skipped for that pass?	
1	How many nonblank records are reviewed?	
1	What final value prints?	

2 Trace break as stop the whole scan

trace stop

A severe damage code should stop the batch scan immediately.

```
checked = 0

for route in ["release", "charge", "repair", "release"]:
    if route == "repair":
        break
    checked = checked + 1

print(checked)
```

Pts.	Prompt	My evidence / response
2	Pass for release: does break happen? checked after pass?	
2	Pass for charge: does break happen? checked after pass?	
2	Pass for repair: does break happen? checked after pass?	
2	Does the final release record get processed? Why or why not?	
2	What final value prints?	

3 Distinguish skipped from stopped

compare explain

Students often mix up continue and break. Use evidence language to separate them.

```
for record in ["A", "missing", "B", "stop", "C"]:
    if record == "missing":
        continue
    if record == "stop":
        break
    print(record)
```

Pts.	Prompt	My evidence / response
2	Which record is skipped but does not stop the scan?	
2	Which record stops the scan?	
2	Which records are printed?	
2	Why is C not printed?	
2	Write one sentence that contrasts continue and break.	

4 Debug wrong loop-control placement

debug test

The intended rule is to count all nonblank records until STOP appears. This version has a likely bug.

```
count = 0
for code in ["A", "", "B", "STOP", "C"]:
    if code == "":
        break
    if code == "STOP":
        break
    count = count + 1
print(count)
```

Pts.	Prompt	My evidence / response
2	What final count does this wrong version print?	
2	What final count should print under the intended rule?	
2	Which break should be a continue?	
2	Write the minimal repair.	
2	What test case would catch the repaired behavior?	

Common mistake to avoid

Blank or missing data often means skip this record, not stop the entire scan.

5 Choose break, continue, or normal pass

decide implement

Choose the loop-control move for each checkout scan rule.

Pts.	Prompt	My evidence / response
2	Rule: blank status should not be counted, but later records still matter. Choose break, continue, or normal pass.	
2	Rule: safety shutdown code appears; no later records should be processed. Choose break, continue, or normal pass.	
2	Rule: route is release; count it and keep scanning. Choose break, continue, or normal pass.	
2	Write one line of Python for the blank-status rule.	
2	Write one line of Python for the shutdown rule.	

6 Transfer and support route

transfer reflect

Close Day 13 by explaining loop-control evidence in a way that avoids hidden skipped work.

Pts.	Prompt	My evidence / response
4	Write a short note explaining how continue and break change which loop statements run.	
2	Circle your support route: current record / skipped code / early stop / final count / expected-vs-actual. Name one next action.	

Support route

Day 14 nests loops: the trace has an outer loop and an inner loop, so the evidence has two loop variables to manage.